

Class 6: R functions

Hailey Phan (PID: A19113445)

Table of contents

Background	1
A first funtion	1
A <code>generate_dna()</code> function	3
Write a <code>generate_protein()</code> function	6
Generate random protein sequences of length 6 to 13	6
BLASTp search against nr — are your peptides “unique in nature”?	7
Connecting your findings to immunology (MHC class II and T-cell activation)	8

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of the R code that do the work of the function)

A first funtion

Here we will create a function to add some numbers. Lets call it `add()`

Input arguments can be either **required** or **optional**. The later have fall-back default values that will be used if the user does not specify them.

```
add <- function (x, y = 0) {  
  x + y  
}
```

Can we use our new function:

Q1a. Your first version of the function should add two input numbers together. For example, `add(4, 7)` should return 11. [1 pt]

```
add(4, 7)
```

```
[1] 11
```

```
add(10)
```

```
[1] 10
```

Q1b. For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add(c(4,7,10))`. [1 pt]

```
add <- function (x, y=0) {  
  sum(x, y)  
}  
add(4, 7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the `...` (dots) argument or the base R function `sum()`. [2 pts]

```
add <- function(...) {  
  # combine all inputs and add them  
  total <- sum(...)  
  return(total)  
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add(c(4, 7, 10))
```

```
[1] 21
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call.

```
add <- function (x, y=0, z = 0) {  
  return(sum(x, y, z))  
  cat("Is it break time yet?\n")  
}  
  
add(10,100)
```

```
[1] 110
```

A `generate_dna()` function

A useful function here is the “base R” `sample()` function:

```
sample(1:5, size = 3)
```

```
[1] 3 2 5
```

```
sample(1:5, size = 6, replace = TRUE)
```

```
[1] 1 5 1 2 2 3
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

```
sample(x = c("A", "C", "G", "T"), size = 10, replace = TRUE)
```

```
[1] "C" "C" "A" "T" "C" "G" "T" "G" "G" "C"
```

Q2a. Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna <- function(length) {
  sample(x = c("A", "C", "G", "T"), size = length, replace = TRUE)
}
```

```
generate_dna(length = 100)
```

```
[1] "T" "G" "G" "G" "A" "A" "C" "T" "G" "T" "C" "A" "T" "G" "A" "A" "C" "C"
[19] "A" "C" "G" "A" "T" "A" "A" "A" "A" "G" "A" "A" "A" "C" "A" "G" "T" "A"
[37] "A" "A" "G" "G" "A" "C" "C" "T" "C" "G" "A" "C" "C" "C" "T" "G" "T" "C"
[55] "A" "C" "A" "G" "G" "G" "C" "A" "T" "G" "C" "A" "T" "T" "C" "G" "A" "C"
[73] "T" "T" "G" "A" "G" "C" "A" "T" "C" "G" "G" "G" "C" "C" "T" "T" "A" "C"
[91] "C" "G" "G" "T" "A" "A" "C" "A" "C" "A"
```

Q2b. Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a *single character string* (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

Functions that could be useful here are `paste()`, `if()`, `cat()`, `return()`

```
# generates random DNA and can return either a vector of bases or one single DNA string
generate_dna <- function(length, single.element = TRUE) {

  # randomly sample DNA bases
  ans <- sample(x = c("A", "C", "G", "T"),
               size = length,
               replace = TRUE)

  # if TRUE, combine into one string
  if (single.element) {
    return(paste(ans, collapse = ""))
  } else {
```

```

    # otherwise return as vector
    return(ans)
  }
}

```

```
generate_dna(6)
```

```
[1] "TATAGA"
```

```
generate_dna(6, single.element = TRUE)
```

```
[1] "GGATAA"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. Be sure to include clear comments that explain each step of your function. [2 pts]

```

# prints DNA in FASTA format
generate_dna <- function(length) {

  # generate random DNA bases
  ans <- sample(x = c("A", "C", "G", "T"),
               size = length,
               replace = TRUE)

  # combine all bases into one sequence
  dna_string <- paste(ans, collapse = "")

  # print FASTA header with sequence length
  cat(">len", length, "\n", sep = "")

  # print DNA sequence
  cat(dna_string, "\n", sep = "")
}

```

```
generate_dna(11)
```

```

>len11
GGAGTAACCTC

```

Write a `generate_protein()` function

Q3. Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return “WQRTAG”. Your function should use the single-letter code for all 20 standard amino acids and no other letters (see earlier list at the beginning of this handout) and include clear comments that explain each step of your function. [2 pts]

```
# This function generates a random protein sequence of the length chosen by the user
generate_protein <- function(length) {

  # create a vector with the 20 standard amino acid single-letter codes
  amino_acids <- c("A", "R", "N", "D", "C",
                  "Q", "E", "G", "H", "I",
                  "L", "K", "M", "F", "P",
                  "S", "T", "W", "Y", "V")

  # randomly sample amino acids with replacement
  protein <- sample(x = amino_acids,
                   size = length,
                   replace = TRUE)

  # combine the letters into one single string
  protein_string <- paste(protein, collapse = "")

  # return the protein sequence
  return(protein_string)
}
```

```
generate_protein(6)
```

```
[1] "HTWHGI"
```

Generate random protein sequences of length 6 to 13

Q4. Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand. Be sure to format your results in FASTA format ready to paste or upload as a query to NCBI BLASTp.

Hint: the function `cat()` combined with `paste()` and the newline character “`\n`” makes this relatively straightforward. [2 pts]

```
# This loop generates random protein sequences from length 6 to 13 in FASTA format
for (i in 6:13) {

  # print FASTA header
  cat(">", i, "\n", sep = "")

  # print one protein sequence of length i
  cat(generate_protein(i), "\n", sep = "")
}
```

```
>6
MIKLLR
>7
GAPATNM
>8
WQHKTFFR
>9
FQQENVPEL
>10
YVGVNFSDQF
>11
EMPVCAYSAGN
>12
FVKMQRQYGWDF
>13
QKSQGHFEINAAW
```

BLASTp search against nr — are your peptides “unique in nature”?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched). In your quarto report create a fill in table along the lines of the following: [2 pts]

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6	100	100	N
7	100	100	N

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
8	100	88	Y
9	88.89	100	Y
10	100	80	Y
11	90	91	Y
12	83.33	100	Y
13	81.82	85	Y

Q5a. At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)? [1 pt]

My randomly generated peptides start to look unique in nature at a length of 8.

Q5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide length of length L) and to the size of the known protein universe. [1 pt]

Very short random peptides are almost always found in nr while longer ones are typically not because for short peptides, many possible sequences are likely already represented somewhere in nr just by chance due to its small sequences. With longer peptides however, the number of possible sequences becomes so large that most random sequences have probably never occurred in known proteins.

Connecting your findings to immunology (MHC class II and T-cell activation)

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a *bad* design choice for the immune system to present very short peptides? [3 pt]

From my Q5 results, the peptides started to seem unique once they reached about 8 amino acids, so I think the minimum length for MHC class II presentation is probably around 8 amino acids or a little longer. Using very short peptides would be a poor design choice because short sequences are much more likely to also appear in the body’s own proteins, which could make it harder for T cells to tell the difference between self and foreign molecules. Once the peptides get longer, the number of possible sequences increases extremely fast (20^L), so the chance of a random foreign peptide matching something already common in the known protein universe becomes much lower. This makes slightly longer peptides much more useful for helping the immune system recognize real pathogens while avoiding unnecessary responses against self proteins.